Baseline RAG vs. Cross-Encoder Re-Ranking

Two-stage retrieval on the IIT Madras B.Tech Ordinances & Regulations · Shravani K (2023115009) · Code: `rag-assignment/`

0. Setup and evaluation

The knowledge source is one 25-page PDF: the IIT Madras *Ordinances and Regulations for B.Tech / B.Tech (Honours)*, 2010–2014 batches. It suits a stress test because it has three near-identical credit tables for different batches, rules with "however"/"except" clauses, prohibitions, and one internal contradiction. Text was extracted with PyMuPDF (tables kept as Markdown rows, captions such as "(for 2013 batch onwards)" kept attached to their table) and split with LangChain's `RecursiveCharacterTextSplitter` into **85 chunks** of up to 1000 characters with 200 overlap, so a clause usually stays together with its exception. Both pipelines use exactly this corpus.

Component	Choice
Bi-Encoder + vector store	`all-MiniLM-L6-v2` (384-d, L2-normalised), FAISS `IndexFlatIP` (inner product = cosine), top-10
Cross-Encoder	`cross-encoder/ms-marco-MiniLM-L-6-v2`, scores the same 10 (query, chunk) pairs
LLM	Gemini 2.5 Flash via Vertex AI; temperature 0, top-p 1, seed 42; one prompt template for both pipelines
Hardware	Intel i5-1235U laptop CPU, 15.3 GB RAM, no GPU

Pipeline 1 sends the top-3 cosine results to Gemini (Answer 1). **Pipeline 2** re-scores the *same* top-10 with the Cross-Encoder and sends its new top-3 (Answer 2). The vector search runs once per query, and automated checks confirmed that the top-10, model, settings and prompt text were identical, so only the three context chunks differed.

Evaluation set. 33 questions, each written for one failure type: 6 keyword traps, 6 exceptions, 6 negations, 6 synonym mismatches, 5 document-specific cases and 4 easy controls. The supporting sentences were copied verbatim from the PDF, and the chunks containing them were marked relevant. Answers were read against a reference answer and labelled *correct*, *partially correct* (a condition missing) or *insufficient context* (the model said the context did not answer).

> **Recall@3**: share of relevant chunks in the top-3 sent to the LLM. **Hit@3**: at least one relevant chunk in the top-3. **MRR@10**: 1 / rank of the first relevant chunk (0 if none in top-10). **Evidence coverage@3**: share of evidence quotes present in the top-3. **Rescue**: a relevant chunk outside the baseline top-3 but inside the top-10 that the Cross-Encoder moves into the top-3; a **regression** is the reverse.

1. Architecture: Bi-Encoder vs. Cross-Encoder

A **Bi-Encoder** encodes the query and each chunk *separately* into one vector, and relevance is the cosine between them. Since a chunk's vector does not depend on the query, all chunks are embedded once offline (85 chunks ≈ 2.2 s), and a query costs one embedding (≈ 10 ms) plus a vector search (0.07 ms). The cost is that each text is compressed into a single vector before the two meet, so the model measures overall topical similarity and cannot check whether a specific word in the query is answered by a specific phrase in the chunk.

A **Cross-Encoder** feeds query and chunk together, `[CLS] query [SEP] chunk`, through the transformer. Self-attention lets every query token attend to every chunk token, so it can tell "opt for Honours" apart from "maintain CGPA 8.5 for the award". Its output is an unbounded relevance logit, so only *ranks* can be compared with the cosine. Nothing can be precomputed: each pair needs its own forward pass at query time.

This explains the failure types in the assignment. When a whole chunk is averaged into one vector, short words such as "not", "except" or "2013" barely change it, so "a course cannot be repeated" and "a course must be repeated" end up close together, as do three tables that differ in one cell. A Cross-Encoder reads both texts jointly and can check that the exact prohibition or batch is present.

	Bi-Encoder	Cross-Encoder
Input / interaction	encoded separately; one cosine at the end	encoded together; token-level self-attention
Precompute chunks?	yes, once, offline	no, every pair at query time
Cost per query (here)	≈ 10 ms total	≈ 25–30 ms per candidate pair
Score / best use	cosine in [−1, 1]; fast recall over the whole collection	unbounded logit; precise ordering of a short list

KT-02 ("What CGPA is needed to *opt* for B.Tech (Honours), and when?") shows the difference. Chunk 84 (R.30.1, keeping CGPA 8.5 to be *awarded* Honours) shares almost all keywords with the true answer, chunk 21 (R.1.12), and outranks it by cosine. The Cross-Encoder reverses this:

Chunk	Content	Cosine	Base rank	CE logit	New rank
78	NCC / NSO rules (mentions Honours)	0.750	1	2.37	3
84	R.30.1: maintain CGPA 8.5 for the *award* (distractor)	0.723	2	1.96	5
33	Honours credit requirements	0.706	3	1.83	6
21	**R.1.12: CGPA ≥ 8.5 after sem. 3, opt during sem. 4**	0.673	4	4.33	1
12	O.5: opt "at the end of the fourth semester"	0.638	8	4.14	2

2. Retrieval results and rank-shift analysis

Pipeline (33 questions)	Recall@3	Hit@3	MRR@10	Evidence coverage@3
Baseline (Bi-Encoder top-3)	0.818	0.848	0.801	0.836
Re-ranked (Cross-Encoder top-3)	**0.879**	**0.909**	**0.863**	**0.894**

Recall@10 (0.939) is the same for both pipelines because they share the candidate list. Per question, 7 improved, 22 were unchanged and 4 worsened, but only 4 improvements changed which evidence reached the LLM (the rest were moves inside the top-3 or from #9 to #8). The meaningful result is **4 rescues and 2 regressions**:

Case	Failure type	Relevant chunk	Bi rank	CE rank	Top-3 before → after	Answer 1 → Answer 2
KT-02	Keyword trap	21 (R.1.12)	4	1	[78, 84, 33] → [21, 12, 78]	declined → correct
EX-05	Exception	61 / 62	1 / 5	1 / 2	[61, 60, 58] → [61, 62, 60]	clause omitted → correct
NG-06	Negation	76 (R.25.1 tail)	4	1	[25, 75, 46] → [76, 59, 75]	wrong "no rule" → correct
DS-02	Near-identical tables	31 (2013+ table)	7	1	[33, 30, 32] → [31, 33, 29]	declined → correct
SY-03	Synonym (regression)	75 / 74	1 / 2	3 / 4	[75, 74, 59] → [48, 72, 75]	correct → declined
DS-05	Table lookup (regression)	58	3	5	[60, 61, 58] → [60, 59, 61]	correct → declined

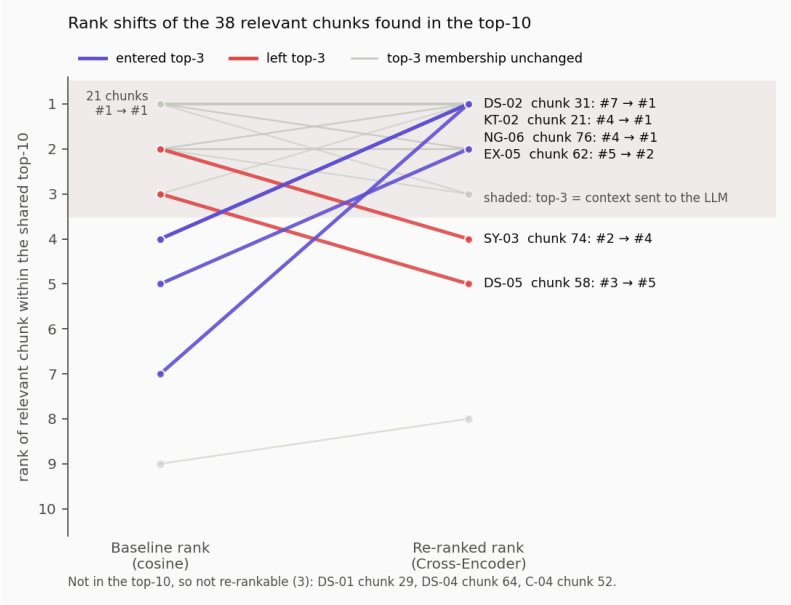


Figure 1. Rank of each relevant chunk in the shared top-10, before and after re-ranking. The shaded band is the top-3 sent to Gemini.

Keyword trap: KT-02

The baseline filled two slots with chunks that repeat "B.Tech (Honours)" and "CGPA 8.5" but describe the award rule, so Gemini answered the wrong rule and admitted the question was not covered:

> **Answer 1:** "To be awarded the B.Tech (Honours) degree, students must maintain an average CGPA of 8.5 [Chunk 84] ... The provided context does not specify when a student can opt for the B.Tech (Honours) programme ..."
> **Answer 2:** "A student needs a CGPA of 8.5 and above at the end of their third semester to opt for the B.Tech (Honours) programme [Chunk 21] ... They can opt for this programme during the fourth semester [Chunk 21]."

Re-ranking also promoted chunk 12 (O.5, #8 → #2), which says "at the *end* of the fourth semester" and conflicts with R.1.12's "during". In two of five runs Answer 2 reported both clauses, exposing a real inconsistency in the document.

Exception: EX-05

"Can attendance below 85% be condoned for a hospitalised student?" The rule has two conditions: actual attendance above 40% *and* "notional" attendance (excluding illness days) of at least 85%. The second sits in chunk 62, ranked #5 by cosine; the attendance-code table took the third slot. Answer 1 gave only the 40% condition, a confident but incomplete rule that would wrongly approve a student meeting just the first condition. This is the most dangerous failure in the set because the answer looks complete. With chunk 62 at #2, Answer 2 stated that "this notional percentage must be ≥ 85% for condonation to be granted".

Negation: NG-06

"Which courses is a student not allowed to repeat?" The answer is one sentence in chunk 76 ("A course successfully completed cannot be repeated", plus a 2014-15 exception for students below CGPA 5). The bi-encoder ranked first chunk 25, which contains an unrelated negation ("No withdrawal ... will be allowed"), followed by U/W repetition rules and the (n − 4) registration rule. The Cross-Encoder gave chunk 25 a logit of −0.26 and moved chunk 76 from #4 to #1 (logit 7.29). Answer 1, seeing only U/W rules, concluded that "there are no courses explicitly stated as non-repeatable": true of its context, but the opposite of the real rule. Answer 2 gave the prohibition and its exception. This case came from screening 24 negation phrasings on retrieval; NG-01 to NG-05, written in advance, did not need rescuing.

Document-specific: DS-02

The three credit tables (2010, 2011–12, 2013+) differ in a few cells. For "HPF credits from 2013", the cosine top-3 held a text paragraph and two table halves without the HPF row; the correct table head (chunk 31) was #7. Answer 1 said HPF was not mentioned. With chunk 31 at #1, Answer 2 answered "4 HPF credits" and did not confuse it with the 2011–12 value (2), which was also in its context.

Where re-ranking hurt

In **SY-03** the user says "dispute a grade" while the document says "feels aggrieved … contact the teacher within three weeks". The Cross-Encoder gave every candidate a strongly negative logit (−5.4 to −10.7), so its ordering was near noise, and chunk 74 with "three weeks" fell to #4. In **DS-05** the answer ("G" for 90% attendance) is in a small table of bare numbers and codes (chunk 58), and prose about the 85% rule was ranked above it. Both fit a known weakness: the model was trained on MS MARCO web passages and handles fluent prose better than tables or paraphrased legal wording. In both cases Answer 2 declined instead of guessing.

> **Answer 1 vs. Answer 2.** In every rescue the critical chunk sat at rank 4–7. The baseline **never hallucinated**: it either **omitted an essential clause** (EX-05) or **answered a neighbouring rule and then declined** (KT-02, NG-06, DS-02). The prompt's instruction to say when evidence is missing likely turned would-be fabrications into refusals, so the practical risk was omission. With the re-ranked context all four answers were correct. One citation slip was found (NG-06 Answer 2 cites p. 22 for a clause on p. 23 of a two-page chunk). **Stability:** each of the six cases was answered 5 times per pipeline (60 calls); the verdict was identical in all five runs for every case, and an automatic key-fact check agreed with every manual verdict.

3. Latency profiling

Components were timed with `time.perf_counter()` after warm-up: retrieval on 33 questions × 5 repeats, generation on 15 paired runs. Model loading and indexing are excluded; 6 rate-limited API attempts were retried and not counted.

Component	n	Mean (ms)	Median (ms)
Query embedding (Bi-Encoder)	165	10.2	10.0
FAISS top-10 search (85 vectors)	165	0.07	0.07
Cross-Encoder scoring (10 pairs)	165	265.0	253.3
Sort by CE score + top-3	165	0.02	0.02
Pipeline 1 retrieval side	165	**10.3**	**10.0**
Pipeline 2 retrieval side	165	**275.4**	**264.0**
Gemini generation, Answer 1 / Answer 2	15	4465 / 5746	4474 / 3256
Pipeline 1 / Pipeline 2 end-to-end	15	**4496 / 6035**	**4510 / 3505**

Re-ranking adds about **265 ms** per query, making the retrieval side roughly **27× slower**; almost all of it is the ten Cross-Encoder forward passes. End to end, Gemini's 3–4.5 s dominates and re-ranking is under 5% of Pipeline 2's total. The end-to-end ratios point in opposite directions (P2 faster by median, slower by mean) because one Answer 2 call took 26 s and Answer 1 alone ranged 1.8–9.2 s; network variance is far larger than the re-ranking cost, so the end-to-end difference is not statistically resolved. A GPU would shrink the Cross-Encoder step; a larger candidate pool would grow it linearly.

4. Scaling: why not run the Cross-Encoder over everything?

The Cross-Encoder needs one forward pass per (query, chunk) pair, and none of it can be cached because each pass depends on the query. Measured here, cost grows linearly: about 31 ms for 1 pair, 232 ms for 10, and **2.68 s for all 85 chunks**, already close to a whole Gemini call. Extrapolating (≈ 32 ms per pair) gives about **32 s per query for 1,000 chunks, 5 min for 10,000 and 53 min for 100,000**. The Bi-Encoder moves the expensive work offline: an exact FAISS search over 100,000 synthetic vectors still took only **9.8 ms**, and approximate indexes (IVF, HNSW) scale further. At 100,000 chunks, two-stage retrieval would cost ≈ 10 ms embedding + 10 ms search + 253 ms re-ranking ≈ 0.27 s per query.

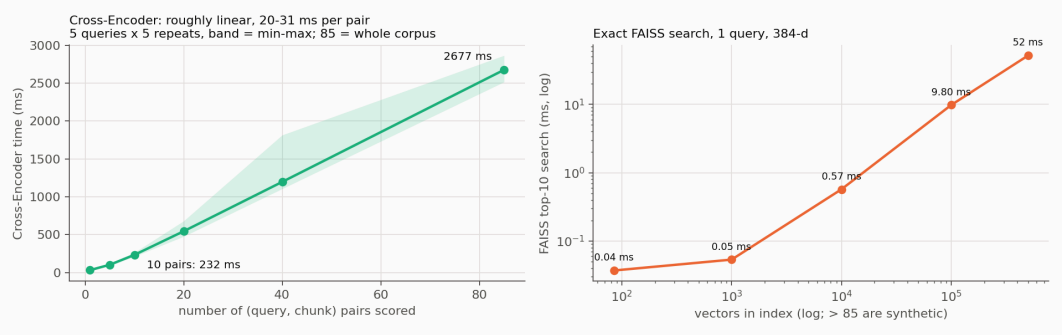


Figure 2. Left: Cross-Encoder time vs number of pairs (measured up to 85, the whole corpus). Right: FAISS exact search vs index size (synthetic vectors above 85).

So the cheap Bi-Encoder narrows the collection to a few plausible candidates, and the precise Cross-Encoder spends its cost only where it changes the answer. The catch is that re-ranking can only reorder what the first stage found: in C-04 the relevant sentence is the last line of a chunk about contact courses, it never reached the top-10, and both pipelines failed. Recall@10 (0.939) is the ceiling for the whole system, so chunking and k matter as much as the re-ranker.

5. Limitations and conclusion

- **Small evaluation.** 33 questions on one document show a pattern, not statistical significance, and the negation rescue was found by a targeted search.
- **Retrieval ≠ answer quality.** In SY-03 the re-ranked top-3 still held a relevant chunk, yet the answer failed because the deciding sentence was in the other one; metrics alone would miss this.
- **Labels and timing.** Relevance labels are evidence-quote chunks, verdicts are one reviewer's judgement (cross-checked automatically), and timings come from one CPU laptop with network-dependent Gemini latency.

Conclusion. Cross-Encoder re-ranking fixed the failure types it was designed for: a keyword distractor, an exception clause, a negated rule and a near-duplicate table were all rescued from ranks 4–7, and each answer went from declining or omitting a condition to correct. It also caused two regressions, on table data and paraphrased wording, and could not help when evidence was outside the top-10. At about 0.27 s per query on a CPU, it is cheap next to LLM generation, while scoring the whole collection would grow linearly and quickly become impractical. Re-ranking a small candidate set is a good trade-off for precision-critical documents like regulations, provided first-stage recall is monitored.